Core Breach

CENG 454, Game Programming, Spring 2025-2026

Homework 3: Pattern-Driven Systems Prototype

Full Name	Alp Doruk Şengün
Student ID	230444401
GitHub URL	https://github.com/AlpoTheo/CENG454-HW3-AlpDoruk-Sengun-230444401
Submission Date	17 May 2026
Unity Version	Unity 6 (URP, 2D)

1. Gameplay Overview

Core Breach is a small 2D top-down prototype. The player must protect a yellow energy core that sits in the middle of the arena. Red enemies appear at the edges of the screen and walk toward the core. The whole game takes place in one short round.

Each wave behaves differently. Wave 1 enemies walk straight to the core. Wave 2 enemies move side to side while they come in, so they are harder to hit. Wave 3 enemies spiral around the core instead of going straight. This is where I use the Strategy pattern. The EnemyController class is the same for every enemy, but the movement behaviour can be changed per wave.

The player moves with WASD, aims with the mouse, and fires with left click. The weapon is built at the start of the scene as a chain of decorators. The base weapon is wrapped by RapidFireDecorator, and that is wrapped by DoubleShotDecorator. Every click fires two bullets at +/-12 degrees, at two times the base fire rate. Projectiles and enemies are taken from object pools, so they are not created with Instantiate during play.

Goal, win condition, lose condition

Goal. Keep the energy core alive through three waves.
Win. All three waves are cleared (no live enemies and no more waves to spawn). The win panel appears and Time.timeScale is set to zero.
Lose. Core HP becomes zero. The lose panel appears and time stops in the same way.
Length. One run takes around 2 to 4 minutes. The exact time depends on how the player keeps enemies away from the core.

2. Architecture Overview

GameSetup is the class that connects everything at the start of the scene. It builds the weapon decorator chain. It gives the wrapped IWeapon to PlayerShooter. Then it calls WaveManager.Init. After this point, most of the classes talk to each other through C# events. They do not call each other directly. I also kept GameManager small. It only listens to events and shows the win panel or the lose panel.

Folder map

```
Assets/Scripts/
  Interfaces/  IDamageable, IMovementStrategy, IWeapon, IPoolable
  Core/        EnergyCore (Observer publisher)
  Player/      PlayerController, PlayerShooter
  Enemy/       EnemyController (uses IMovementStrategy)
  Strategies/  DirectMoveStrategy, ZigZagMoveStrategy, OrbitMoveStrategy
  Weapons/     BaseWeapon, WeaponDecorator, RapidFireDecorator, DoubleShotDecorator
  Pool/        ObjectPool, Projectile
  Managers/    GameSetup, WaveManager, GameManager
  UI/          HUDController
```

How systems talk

EnergyCore publishes damage events. The TakeDamage method runs when the core takes a hit. Inside this method, OnHealthChanged is called. I send the new health and the max health. If health is zero, I also call OnCoreDestroyed. This happens only one time.

HUDController listens to these events. It uses OnHealthChanged for the HP slider. It uses OnCoreDestroyed to hide the HUD at the end of a run. GameManager also listens to OnCoreDestroyed. It shows the lose panel. So the two listeners do not know about each other. And EnergyCore does not know about them either. I think this is the main point of Observer.

WaveManager has two more events. OnWaveStarted and OnAllWavesCleared. HUDController updates the wave label when OnWaveStarted fires. GameManager shows the win panel when OnAllWavesCleared fires. Inside WaveManager I have a Phase enum with four values. WaitingToStart, WaveActive, BetweenWaves, Finished. Update changes the phase with a timer. EnemyController has a static OnEnemyDied event. WaveManager subscribes to it. So it can count dead enemies. It does not need a list of enemy references.

For player input I use the new UnityEngine.InputSystem package. The PlayerController class reads keyboard state from Keyboard.current. PlayerShooter reads mouse state from Mouse.current. Both of them check for null first, in case the device is missing. Player movement runs in FixedUpdate on a Rigidbody2D for me. Aim rotation and shooting run in Update instead. Projectiles and enemies are pooled too. They reset their own state inside IPoolable.OnSpawn and IPoolable.OnDespawn.

3. Pattern Justification Table

Each row in the table points to specific files in the project. Every pattern solves a real problem. If I remove any of them, the code will need many type checks, or it will have performance problems from Instantiate calls during play.

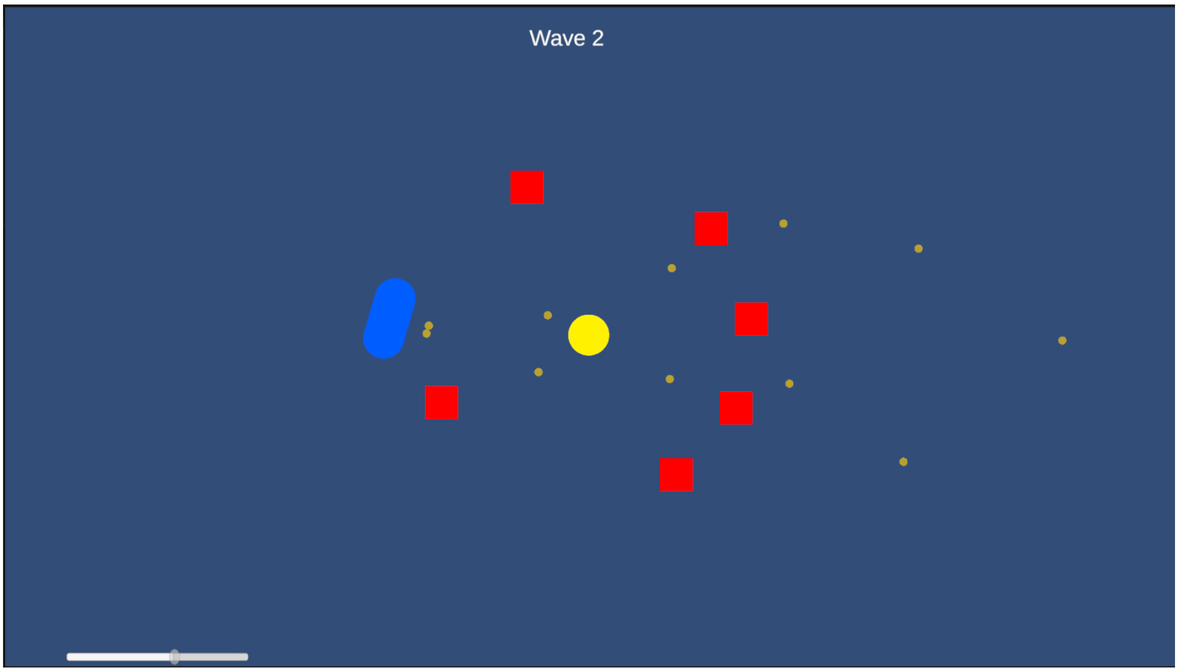
Pattern	Classes / Interfaces Involved	Problem Solved	Why This Choice Was Appropriate
Observer	EnergyCore.OnHealthChanged, EnergyCore.OnCoreDestroyed, WaveManager.OnWaveStarted, WaveManager.OnAllWavesCleared, EnemyController.OnEnemyDied. Listeners: HUDController, GameManager, WaveManager.	Many classes react to one change. Core damaged. Wave ended. Enemy died. With direct method calls, every class needs a reference to EnergyCore.	C# events keep EnergyCore unaware of its listeners. Later I can add an AudioManager that listens to OnHealthChanged. EnergyCore code does not change.
Strategy	IMovementStrategy + DirectMoveStrategy, ZigZagMoveStrategy, OrbitMoveStrategy. Owner class: EnemyController. Selector: WaveManager.SelectStrategyForWave.	Each wave should feel different. For example, Wave 1 walks straight. Wave 2 weaves. Wave 3 spirals. With if-else inside EnemyController, that class would mix two jobs. Enemy logic and wave-tuning logic.	EnemyController only knows the IMovementStrategy interface. For a new movement type, I just write one new class file. Then I add one new case in WaveManager. EnemyController itself does not change. The other strategy classes do not change either. For me this is the biggest benefit of Strategy.
Object Pool	ObjectPool (generic) + IPoolable. Pooled types: Projectile, EnemyController.	Projectiles fire many times in one second. Enemies arrive together in groups of 5 to 9 per wave. If I call Instantiate and Destroy every time, the garbage collector causes frame drops. I noticed this in my early tests, before I added the pool.	The pool reuses GameObjects instead of destroying them after use. IPoolable.OnSpawn and IPoolable.OnDespawn are a place where each class resets its own state. For example, my Projectile class resets its timer and velocity here. So a reused object behaves the same as a fresh one.
Decorator	IWeapon + BaseWeapon + WeaponDecorator (abstract) + RapidFireDecorator + DoubleShotDecorator. Composed in GameSetup.	I want weapon mods to combine at runtime. For example rapid fire, double shot, or piercing later. With subclasses the class count grows fast. I would need Base, Rapid, Double, RapidDouble, and more.	Each decorator wraps an IWeapon. It only overrides what it changes. For example, FireRate or Fire. So I can combine mods at runtime. I do not need a separate subclass for every combination.
Interfaces	IDamageable (EnergyCore, EnemyController), IMovementStrategy (3 strategies), IWeapon (BaseWeapon and decorators), IPoolable (Projectile, EnemyController).	Direct references between classes make changes harder. For example, Projectile should damage any object that can take damage. Not only enemies.	Each interface has at least two implementations. Each one is also a boundary between systems. This is the seam from the brief.

Pattern	Classes / Interfaces Involved	Problem Solved	Why This Choice Was Appropriate
Singleton	Not used.	Not applicable.	At first I thought about Singleton for GameManager. But the other classes talk to it through events. They do not need a global reference. So I kept GameManager as a normal MonoBehaviour. This is simpler for me. Singletons can also cause lifecycle problems in Unity.

What changed during the build. At first I had only two strategies. Direct and ZigZag. After a few Wave 3 runs the movement felt too boring for me. So I wrote OrbitMoveStrategy as a third one. It was a new file plus one new case in SelectStrategyForWave. Nothing else changed. The decorator chain order also changed in the middle. My first version had DoubleShot wrap BaseWeapon. I added RapidFire on top later. But RapidFire feels much better when DoubleShot inherits the faster rate. So I changed the chain order in GameSetup. Now RapidFire is the inner one.

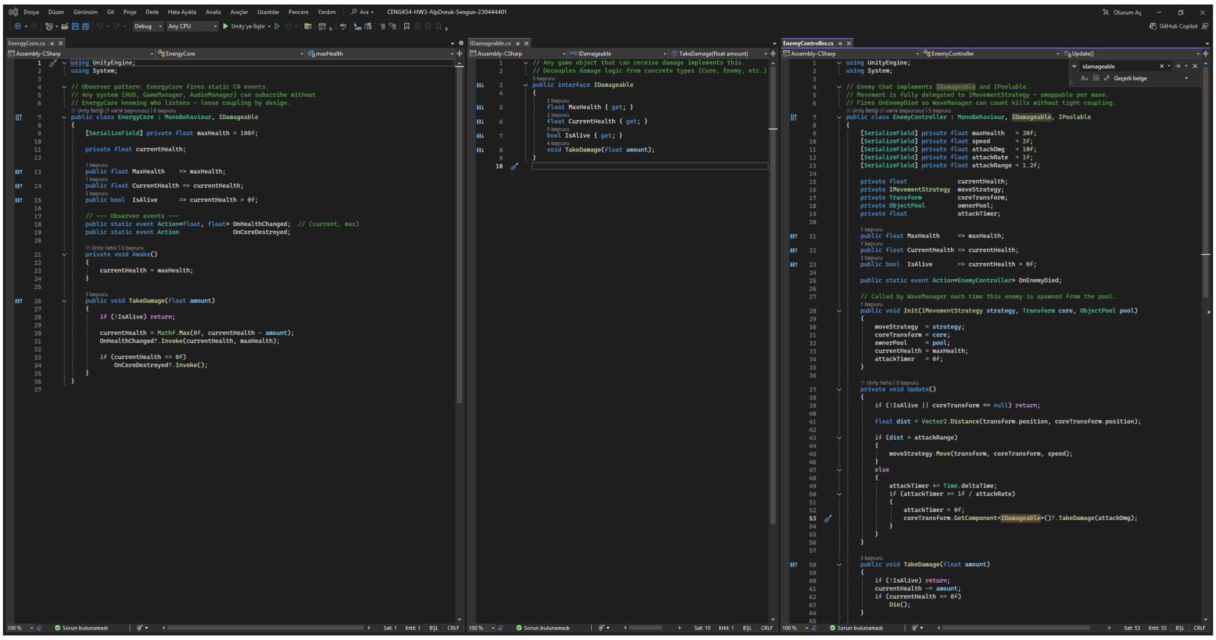
4. Screenshots and Evidence

Screenshot 1: Playable arena and the defendable objective



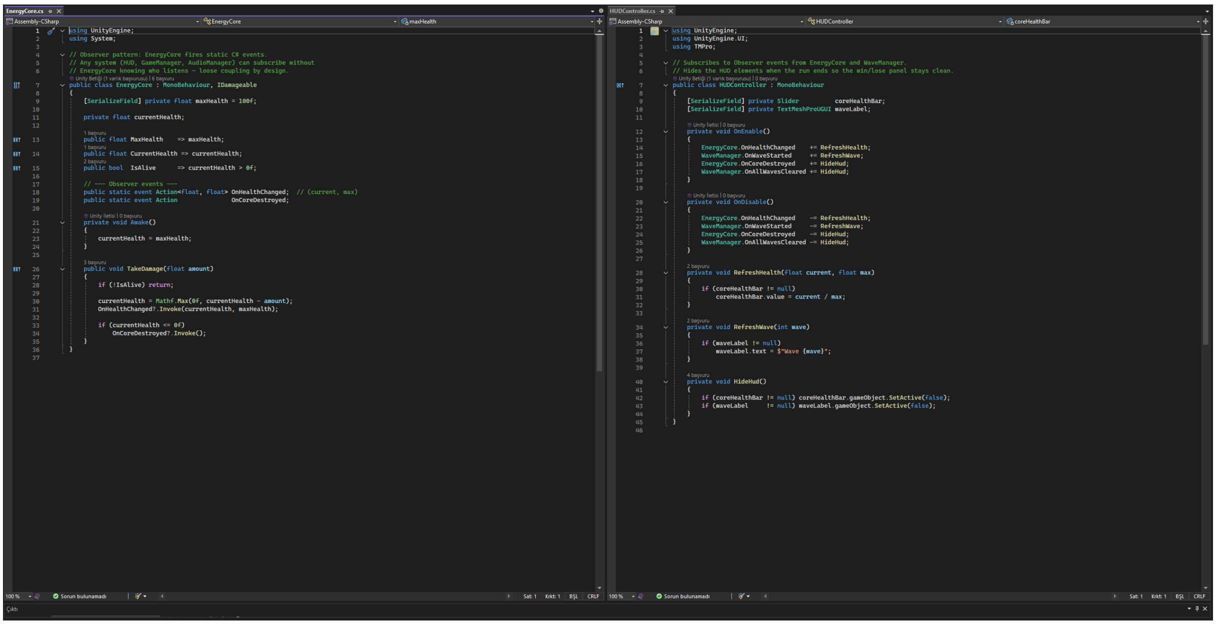
This is Wave 2 in progress. The yellow circle in the middle is the EnergyCore (the objective). The blue capsule is the player I control. Red squares are the enemy units. The small yellow dots flying around are projectiles from the pool. The white slider on the bottom left is the core HP. It is driven by the EnergyCore.OnHealthChanged event.

Screenshot 2: Custom interface with two implementations



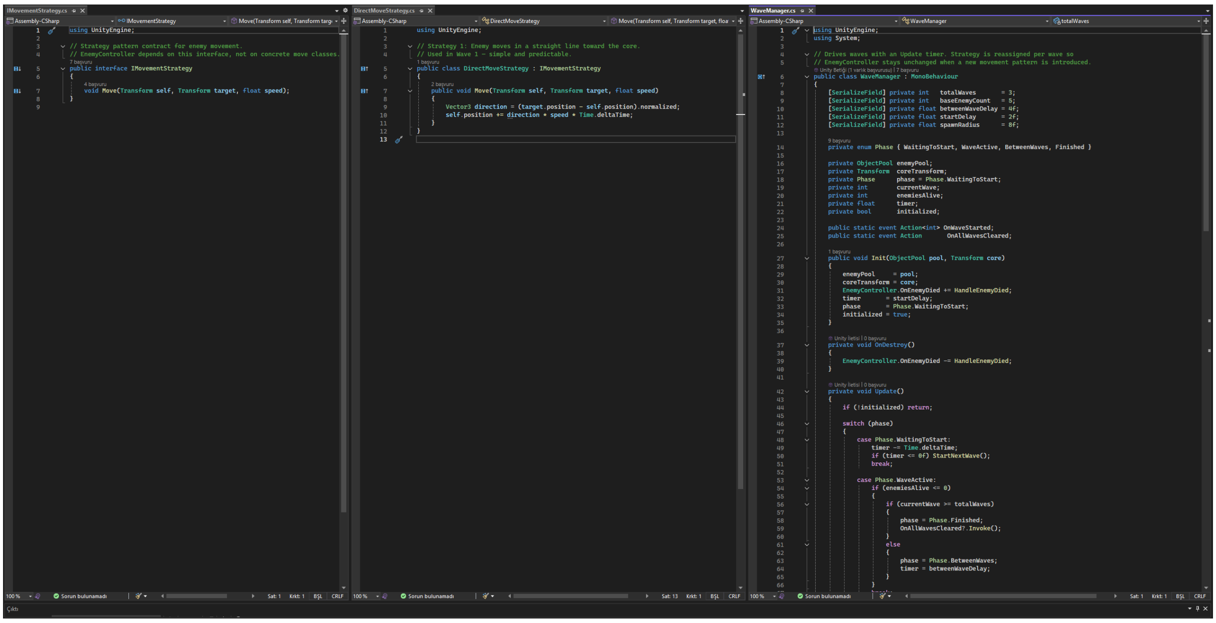
The IDamageable interface is shown on the left side. EnergyCore (which holds the core HP) and EnemyController (which holds the enemy HP) both implement this interface. Inside Projectile.OnTriggerEnter2D, I talk to the interface and never to the concrete classes. So a future destructible barrier could be damaged by the same code without any change.

Screenshot 3: Observer-style event flow



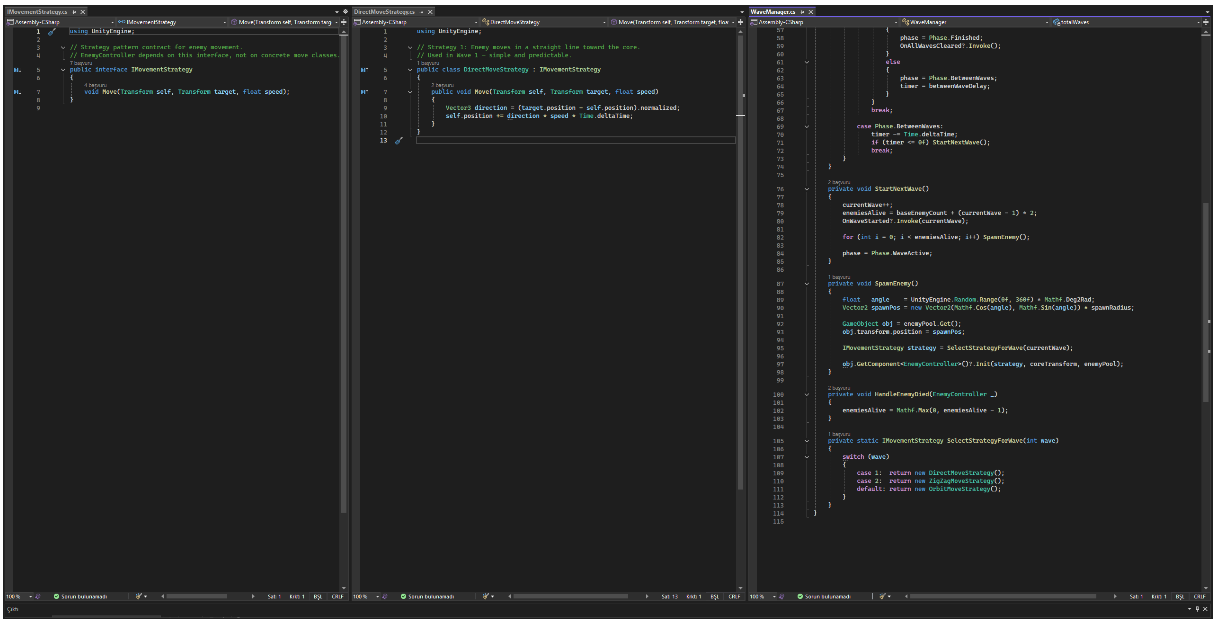
On the left side, EnergyCore raises OnHealthChanged and OnCoreDestroyed inside the TakeDamage method. On the right side, HUDController subscribes in OnEnable. It unsubscribes in OnDisable. GameManager also subscribes to the same OnCoreDestroyed event. But HUDController does not know that GameManager exists. For me this is the main decoupling benefit of Observer.

Screenshot 4a: Strategy contract and concrete movements



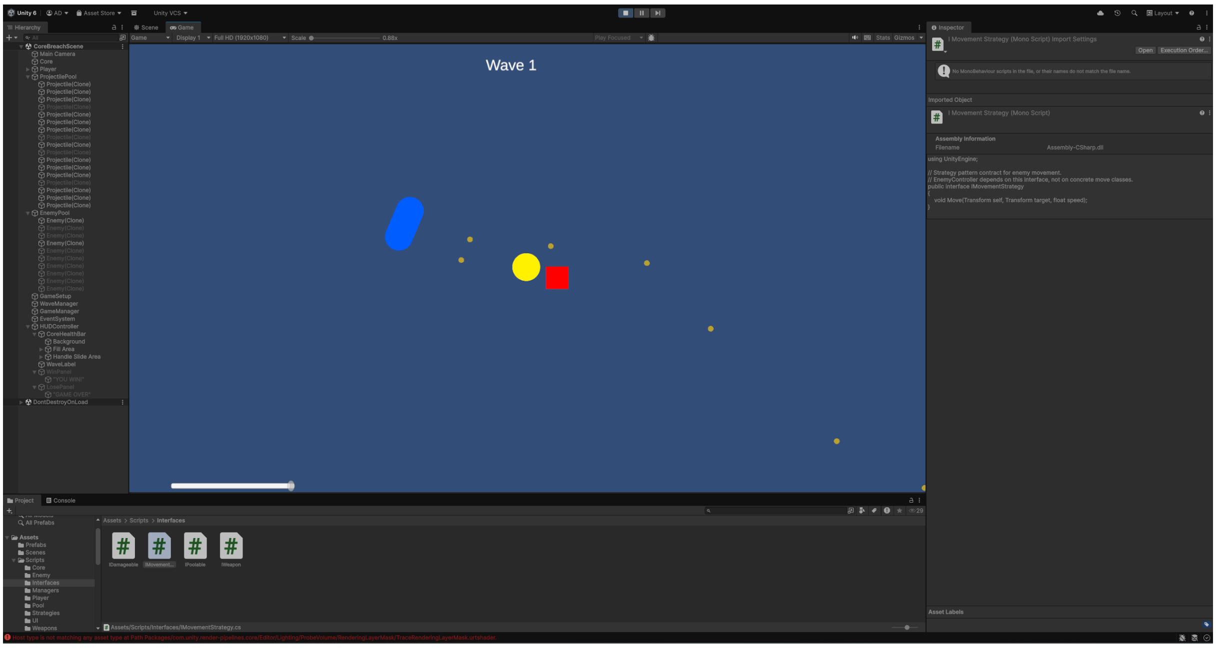
On the left I show the IMovementStrategy interface. It has one method, Move(self, target, speed). DirectMoveStrategy makes the enemy walk straight to the core. ZigZagMoveStrategy uses sin for a weaving motion. OrbitMoveStrategy spirals around the core. The EnemyController class holds an IMovementStrategy field. In Update, it just calls strategy.Move(). I do not check the concrete type anywhere.

Screenshot 4b: Strategy per-wave selection



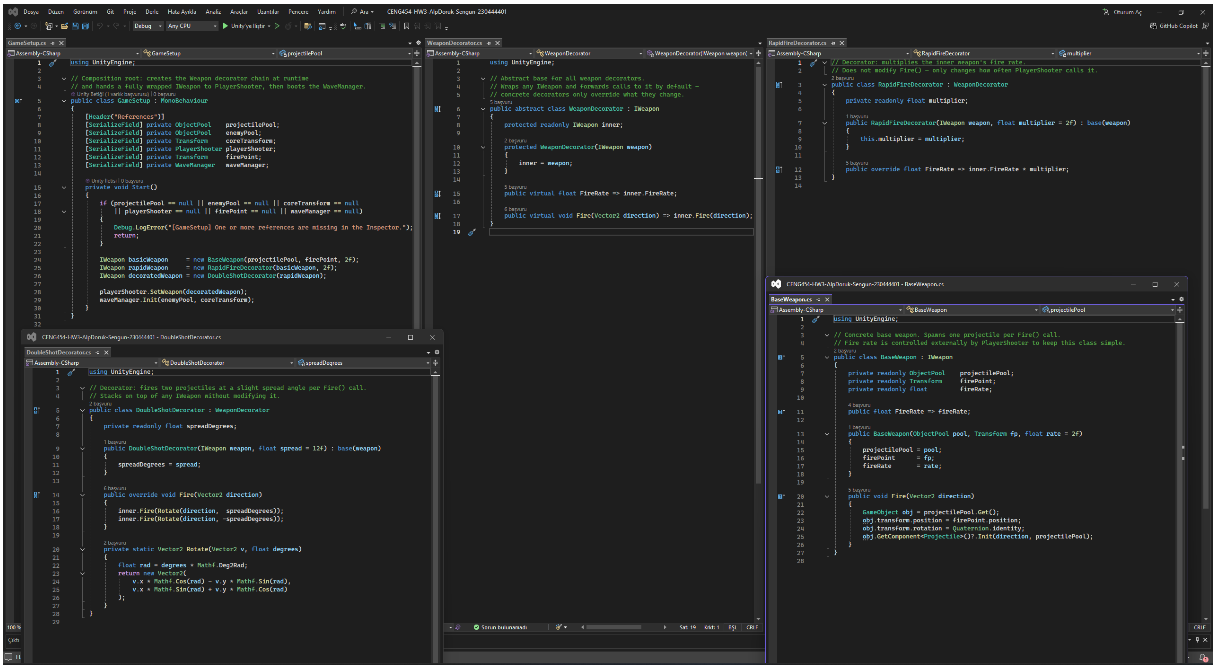
Inside SpawnEnemy, WaveManager calls SelectStrategyForWave with currentWave. The result is passed into EnemyController.Init. Wave 1 picks Direct. Wave 2 picks ZigZag. Wave 3 picks Orbit. If I want to add a fourth strategy later, I only add a new case in this switch statement.

Screenshot 5: Object Pool runtime evidence



The Hierarchy panel is on the left side. ProjectilePool and EnemyPool with their pre-instantiated children underneath. Inactive instances stay under the pool transform. When I call Get(), they become active and move into the world. When I call ReturnToPool, they become inactive again and go back under the pool. The right side shows the Inspector during Wave 1 in play mode.

Screenshot 6a: Decorator (IWeapon and chain classes)



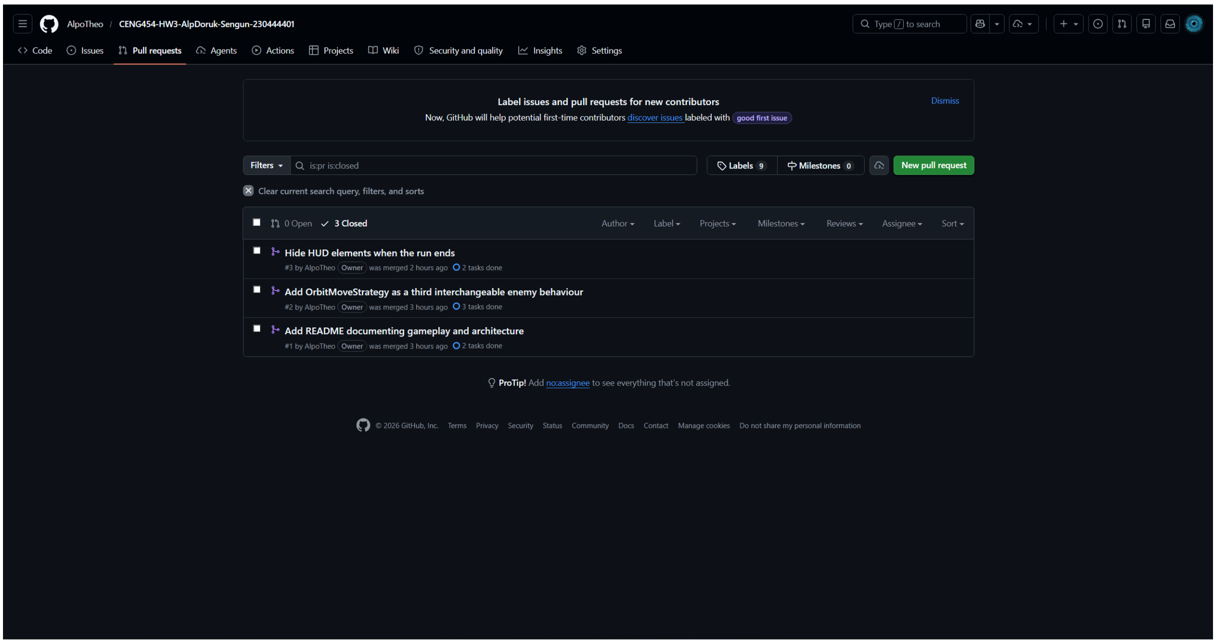
IWeapon is the interface. BaseWeapon is the concrete leaf class. It uses ObjectPool to spawn a projectile. WeaponDecorator is the abstract base for all decorators. It holds an inner IWeapon and forwards the calls by default. RapidFireDecorator only overrides FireRate. DoubleShotDecorator only overrides Fire. Inside Fire, it uses a small rotation helper to send two projectiles at +/-12 degrees.

Screenshot 6b: Decorator composition root

```csharp
using UnityEngine;

// Composition root: creates the Weapon decorator chain at runtime
// and hands a fully wrapped IWeapon to PlayerShooter, then boots the WaveManager.
public class GameSetup : MonoBehaviour
{
    [Header("References")]
    [SerializeField] private ObjectPool    projectilePool;
    [SerializeField] private ObjectPool    enemyPool;
    [SerializeField] private Transform     coreTransform;
    [SerializeField] private PlayerShooter playerShooter;
    [SerializeField] private Transform     firePoint;
    [SerializeField] private WaveManager   waveManager;

    private void Start()
    {
        if (projectilePool == null || enemyPool == null || coreTransform == null
            || playerShooter == null || firePoint == null || waveManager == null)
        {
            Debug.LogError("[GameSetup] One or more references are missing in the Inspector.");
            return;
        }

        IWeapon basicWeapon     = new BaseWeapon(projectilePool, firePoint, 2f);
        IWeapon rapidWeapon     = new RapidFireDecorator(basicWeapon, 2f);
        IWeapon decoratedWeapon = new DoubleShotDecorator(rapidWeapon);

        playerShooter.SetWeapon(decoratedWeapon);
        waveManager.Init(enemyPool, coreTransform);
    }
}
```

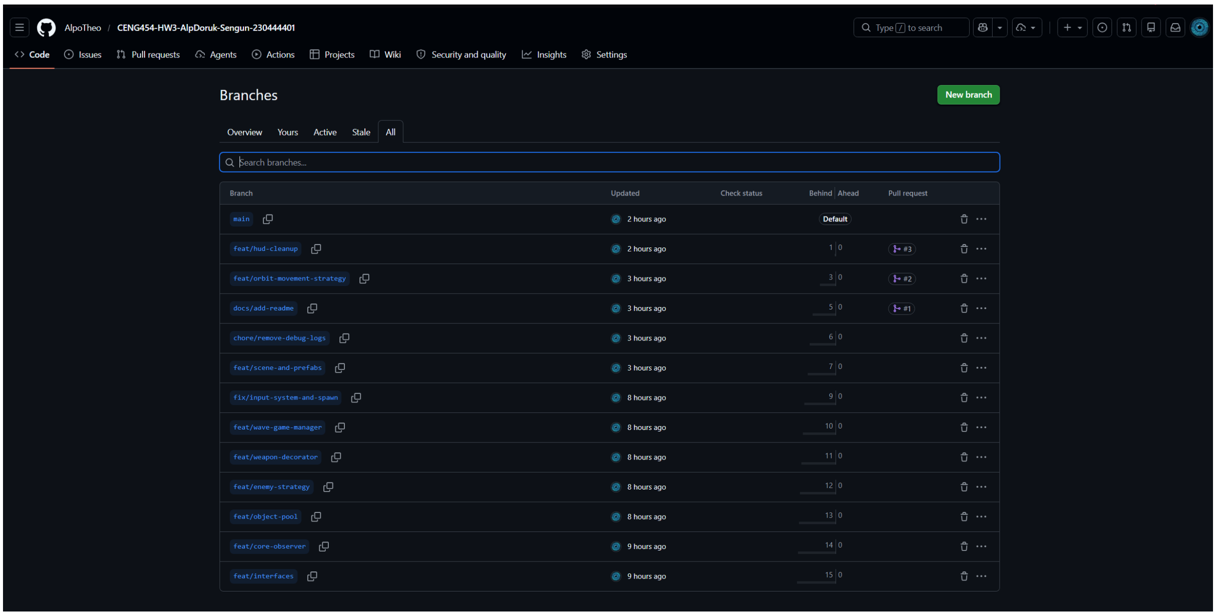
GameSetup.Start builds the chain in just three lines. First a BaseWeapon. Then it is wrapped by RapidFireDecorator with a 2x multiplier. Then by DoubleShotDecorator on top. The fully wrapped IWeapon is given to PlayerShooter through the SetWeapon method. PlayerShooter holds an IWeapon, not a concrete class. So I can change the chain order without touching PlayerShooter.

Screenshot 7a: Repository merged pull requests



Three closed pull requests on main: docs/add-readme (#1), feat/orbit-movement-strategy (#2), feat/hud-cleanup (#3). Each PR was opened from a feature branch and merged after testing the change in Play mode. The minimum is two merged PRs, so I have one extra.

Screenshot 7b: Repository feature branches



This is the branches page on GitHub. The main branch holds the stable code. There are 12 more feature branches around it. The prefix on each branch tells me the kind of work. feat is for a new feature. fix is for a bug fix. chore is for cleanup. docs is for the README. Each branch carries one small slice of work. So I can review history commit by commit.

5. Debugging Story

Bug: The WaveManager did not start the first wave. I pressed Play, but no enemies appeared on the screen.

Setup

I had just finished the first version of my WaveManager. I added all the Inspector references in GameSetup. Then I pressed Play. The scene loaded fine. The player could move with WASD. The HP slider was visible at the bottom. But the wave label stayed empty. No enemy spawned on the screen. And there was no error in the Console either. For me this kind of bug is harder than a normal exception. There is no error message to read. I do not know where to start looking first.

Symptoms observed

The Console showed my `[GameSetup] Start` log. So I knew that GameSetup.Start ran without error. It also called WaveManager.Init successfully. After the two-second start delay, my `[WaveManager] StartWave` log should appear next. But it never appeared. No enemy spawned. No exception was thrown. The WaveManager GameObject was active and enabled in the Inspector. So WaveManager was alive and ready, but it did nothing for some reason.

Diagnosis

My first version of WaveManager used a coroutine for the timing. The Init method called StartCoroutine on a private IEnumerator named RunWaves. The very first line of RunWaves was `yield return new WaitForSeconds(startDelay)`. To find the bug, I added two Debug.Log lines around the yield. One line right before the yield. Another one right after it. The first log printed in the Console. But the second one never printed at all.

So the coroutine paused at the yield. And it never came back. I tried some small changes. I moved StartCoroutine out of Init. I tried `yield return null` as a test. I checked Time.timeScale too. Nothing worked. There was no error in the Console. The GameObject was active. The script was enabled. The coroutine just refused to resume for some reason.

After about two hours of this, I gave up on the coroutine. I decided to write the timing in a different way. A coroutine hides its state inside IEnumerator. I cannot read it from outside. But an enum field can be `[SerializeField]`. Then I can see the value in the Inspector during play. This is much easier for me to debug.

Fix applied

I removed the coroutine completely. Now I do the timing inside Update. The state is a private Phase enum. It has four values: WaitingToStart, WaveActive, BetweenWaves, and Finished. A float timer counts down on every frame. The code is like this:

```
private enum Phase { WaitingToStart, WaveActive, BetweenWaves, Finished }
private Phase phase = Phase.WaitingToStart;
private float timer;

private void Update()
{
    switch (phase)
    {
        case Phase.WaitingToStart:
```

```
            timer -= Time.deltaTime;
            if (timer <= 0f) StartNextWave();
            break;
        case Phase.WaveActive:
            if (enemiesAlive <= 0) { /* advance */ }
            break;
        case Phase.BetweenWaves:
            timer -= Time.deltaTime;
            if (timer <= 0f) StartNextWave();
            break;
    }
}
```

Outcome

I pressed Play again. The enemies spawned correctly this time. All three waves played in order. The delay between the waves was correct too. OnAllWavesCleared fired one time. It fired right after the last Wave 3 enemy died. While the game was running, I could watch the Phase value change in the Inspector. So I did not need any more Debug.Log lines for this.

What it taught me

Sometimes the best fix is not to repair the broken code. Sometimes I need a different approach. A coroutine hides its state in IEnumerator. I cannot read it from outside. A state machine with an enum is more code. But I can read the value during play. After this bug, I prefer Update with an enum for any timing logic. The one exception is a simple one-time delay.

6. Troubleshooting Answer: Debug Report #003 (Ghost Subscriber in a Pool)

The homework brief describes a hypothetical bug. A pooled enemy is subscribed to a core-damaged event. The enemy dies and goes back to the pool. Later, the enemy is reused. After this, a single core hit creates duplicate reactions.

(a) Visible symptom

My project does not have any audio yet. So I describe how the symptom would look in my own project. The HP slider drops more than one enemy hit should cause. The first thing I would notice is the HP bar moving in big jumps. To prove the bug, I would put a Debug.Log inside the React handler. After the first core hit, the log prints two times. After the next hit, it prints three times. The number gets bigger with every wave. Inactive enemies in the pool are still on the delegate list, even though they are not playing. Later, when I add audio or VFX, the same bug will play duplicate sounds. Or it will stack particle bursts on top of each other.

(b) Root cause

The enemy class subscribes to `EnergyCore.OnCoreDamaged` inside the Awake method. This is the real problem. Awake only runs one time per object. It runs when the GameObject is first created in the scene. When the enemy dies, the pool calls `SetActive(false)` on it. But SetActive(false) does nothing to the C# event subscription. The static delegate still keeps the reference to the inactive enemy. So the next OnCoreDamaged call reaches every enemy on the delegate list. Even the inactive enemies sitting in the pool also receive the callback.

And the problem gets even worse on reuse. The dead enemy goes back into the pool. Some time later, it gets pulled out again for a new wave. Now any subscribe line can run a second time. For example a second Awake call. Or an OnEnable line that also subscribes. Either way, another copy of the same subscription is added to the list. So the same enemy ends up on the delegate list two or three times. After many reuses, the handler runs many times for the same enemy on every core hit.

(c) Exact fix

First, I move the subscription out of Awake. I subscribe in OnEnable instead. And I unsubscribe in OnDisable. ObjectPool already toggles the active state of the object on every Get and ReturnToPool call. So OnDisable runs by itself when the enemy goes back to the pool. And OnEnable runs once when the enemy is reused. The fix is just this:

```
private void OnEnable()  { EnergyCore.OnCoreDamaged += React; }
private void OnDisable() { EnergyCore.OnCoreDamaged -= React; }
```

If a subscribe line must run earlier than OnEnable, for example inside `IPoolable.OnSpawn`, then the matching unsubscribe line must run inside `IPoolable.OnDespawn`. The ObjectPool.ReturnToPool method calls OnDespawn before it puts the object back in the pool. So the cleanup is guaranteed.

(d) Prevention rule

Subscribe and unsubscribe must be in a paired lifecycle hook. The hook must run every time the object becomes active or inactive. For pooled objects, the two safe pairs are (OnEnable, OnDisable) and (IPoolable.OnSpawn, IPoolable.OnDespawn). Awake and OnDestroy are not safe for pooled objects. The pool does not destroy them between uses. So OnDestroy never runs. So the unsubscribe never runs. After this homework, I check every += on a static event. I look for a matching -= in the same class. If it is missing, the class can become a ghost subscriber later.

7. Retrospective

The choice that helped me the most was keeping GameSetup small. GameSetup only builds the weapon chain and starts WaveManager. GameManager is also small. It only listens to events and shows the win or lose panel. No other class has a direct reference to GameManager. Because of this, I could change the panels, the HUD, or the weapon chain without changing other classes. The codebase stayed easy to read at the end of the project.

If I had more time, I would add an IPoolService class. It would hold the pools and return them by name. My current code uses Inspector references, which works for one scene but is harder to use across many scenes. I would also add a Composite for enemy groups on top of WaveManager. The brief lists Composite as an option, and it would fit well next to the IMovementStrategy I already have.

Repository

`https://github.com/AlpoTheo/CENG454-HW3-AlpDoruk-Sengun-230444401`